

MCA First Internal Practical Examination

Paper 1

Question 1: Creation of Git repository, file upload, commit and repository management.

1. **Initialize a local Git repository:**

2. Bash

```
mkdir devops_lab
```

```
cd devops_lab
```

```
git init
```

3.

4.

5. **Configure user details (if not set globally):**

6. Bash

```
git config user.name "Your Name"
```

```
git config user.email "your.email@example.com"
```

7.

8.

9. **Create files (file upload/creation):**

10. Bash

```
echo "Hello DevOps Lab" > index.html
```

```
echo "console.log('DevOps Lab');" > app.js
```

11.

12.

13. **Track and add files to the staging area:**

14. Bash

```
git add index.html app.js
```

15.

16.

17. **Commit the changes:**

18. Bash

```
git commit -m "Initial commit: Added index.html and app.js"
```

19.

20.

21. **Repository management (view status and logs):**

22. Bash

```
git status
```

```
git log --oneline
```

23.

24.

Question 2: Exploring Git Commands through Collaborative Coding – Branching and Merging

1. **Create and switch to a new feature branch:**
2. Bash

```
git checkout -b feature-login
```

- 3.
- 4.
5. **Make changes on the new branch:**
6. Bash

```
echo "function login() { return true; }" >> app.js
```

- 7.
- 8.
9. **Stage and commit changes on the feature branch:**
10. Bash

```
git add app.js
```

```
git commit -m "Add login functionality"
```

- 11.
- 12.
13. **Switch back to the main branch:**
14. Bash

```
git checkout main
```

- 15.
- 16.
17. **Merge the feature branch into main:**
18. Bash

```
git merge feature-login
```

- 19.
- 20.
21. **Delete the merged feature branch:**
22. Bash

```
git branch -d feature-login
```

- 23.
- 24.
25. **Verify branch history:**
26. Bash

```
git log --graph --oneline
```

- 27.
- 28.

Paper 2

Question 1: Demonstrate a practical on Version Control Tools.

1. **Check installed Version Control Tool (Git):**
2. Bash

```
git --version
```

- 3.
- 4.
5. **Create a practical directory and initialize tracking:**
6. Bash

```
mkdir vcs_demo
```

```
cd vcs_demo
```

```
git init
```

- 7.
- 8.
9. **Create a sample document and track changes:**
10. Bash

```
echo "Version 1.0 of Project Documentation" > doc.txt
```

```
git add doc.txt
```

```
git commit -m "Version 1.0 commit"
```

- 11.
- 12.
13. **Modify the document to demonstrate version tracking:**
14. Bash

```
echo "Version 1.1 update added" >> doc.txt
```

```
git diff
```

```
git add doc.txt
```

```
git commit -m "Version 1.1 commit"
```

- 15.
- 16.
17. **Inspect full change history:**
18. Bash

```
git log --stat
```

- 19.

Question 2: Exploring Git Commands through Collaborative Coding – Creating and Committing Changes

1. **Create project files:**
2. Bash

```
echo "<h1>Welcome to Collaborative Project</h1>" > index.html
echo "body { background-color: #f4f4f4; }" > style.css
```

- 3.
- 4.
5. **Check untracked status:**
6. Bash

```
git status
```

- 7.
- 8.
9. **Stage specific files or all files:**
10. Bash

```
git add .
```

- 11.
- 12.
13. **Commit staged changes with a concise message:**
14. Bash

```
git commit -m "Add basic UI files (HTML & CSS)"
```

- 15.
- 16.
17. **Verify commit status:**
18. Bash

```
git status
```

- 19.
- 20.

Paper 3

Question 1: Implementation of Git commands such as git init, git add, git commit and git status.

1. **Initialize repository (git init):**
2. Bash

```
mkdir git_basics
```

```
cd git_basics
```

```
git init
```

- 3.
- 4.

5. **Check status before adding files (git status):**

6. Bash

git status

7.

8.

9. **Create a file and check status:**

10. Bash

echo "System Architecture Document" > arch.txt

git status

11.

12.

13. **Stage the file (git add):**

14. Bash

git add arch.txt

git status

15.

16.

17. **Commit the staged changes (git commit):**

18. Bash

git commit -m "Initial commit: Added arch.txt"

19.

20.

21. **Verify status after commit (git status):**

22. Bash

git status

23.

Question 2: Exploring Git Commands through Collaborative Coding – Setting Up Git Repository

1. **Create and initialize central local workspace:**
2. Bash

```
mkdir project_workspace  
cd project_workspace  
git init
```

- 3.
- 4.
5. **Set up local user configurations for the repository:**
6. Bash

```
git config user.name "Developer One"  
git config user.email "dev1@example.com"
```

- 7.
- 8.
9. **Verify configuration settings:**
10. Bash

```
git config --list --local
```

- 11.
- 12.
13. **Set default main branch name:**
14. Bash

```
git branch -M main
```

- 15.
- 16.
17. **Create initial README file and commit setup:**
18. Bash

```
echo "# Team Project Repository" > README.md  
git add README.md  
git commit -m "Setup repository structure with README"
```

- 19.

MCA Second Internal Practical Examination

Paper 1

Question 1: Implement GitHub Operations using Git – Push/Pull Changes to GitHub.

1. **Link local repository to a remote GitHub repository:**
2. Bash

```
git remote add origin https://github.com/<your-username>/<your-repo-name>.git  
git branch -M main
```

- 3.
- 4.
5. **Push local commits to GitHub:**
6. Bash

```
git push -u origin main
```

- 7.
- 8.
9. **Fetch and merge changes from GitHub (Pull):**
10. Bash

```
git pull origin main
```

- 11.
- 12.
13. **Verify remote links:**
14. Bash

```
git remote -v
```

- 15.
- 16.

Question 2: Create account on Docker step by step.

1. **Step 1:** Open your web browser and navigate to <https://hub.docker.com>.
2. **Step 2:** Click on the **Sign Up** button located in the top-right corner.
3. **Step 3:** Enter your **Docker ID** (a unique username), **Email address**, and **Password**.
4. **Step 4:** Accept the Docker Terms of Service and click **Sign Up**.
5. **Step 5:** Check your email inbox for a verification link sent by Docker and click **Verify Email Address**.
6. **Step 6:** Log in to your Docker Hub account in the browser.
7. **Step 7:** Verify the account connection via CLI:
8. Bash

```
docker login -u <your-docker-id>
```

- 9.
10. *(Enter your password or personal access token when prompted.)*

Paper 2

Question 1: Implement GitHub Operations using Git – Cloning a Repository.

1. **Locate the URL of the GitHub repository to clone.**
2. **Clone the remote repository locally:**
3. Bash

```
git clone https://github.com/<username>/<repository-name>.git
```

- 4.
- 5.
6. **Navigate into the cloned directory:**
7. Bash

```
cd <repository-name>
```

- 8.
- 9.
10. **Verify repository status and remote connection:**
11. Bash

```
git status
```

```
git remote -v
```

- 12.

Question 2: To study Dockerfile instructions, build an image for a sample web application using Dockerfile.

1. Create application files (**app.py** and **requirements.txt**):

app.py:

2. Python

```
from flask import Flask
app = Flask(__name__)
```

```
@app.route('/')
def hello():
    return "Welcome to Dockerized Web Application!"
```

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

- 3.
4. *requirements.txt:*
5. Plaintext

flask

- 6.
- 7.
8. Create the **Dockerfile** with standard instructions:
9. Dockerfile

```
# Use official lightweight Python base image
FROM python:3.9-slim
```

```
# Set working directory inside container
WORKDIR /app
```

```
# Copy dependency list and install
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copy application source code
COPY app.py .
```

```
# Expose port
EXPOSE 5000
```

```
# Command to run application
CMD ["python", "app.py"]
```

- 10.
- 11.

12. Build the Docker image:

13. Bash

```
docker build -t my-web-app:v1 .
```

14.

15.

16. Run container from the built image:

17. Bash

```
docker run -d -p 5000:5000 --name sample_app my-web-app:v1
```

18.

19.

20. Verify running application:

21. Bash

```
curl http://localhost:5000
```

22.

Paper 3

Question 1: Create a merge request on GitLab and Review the merge request.

1. Clone the GitLab repository and create a feature branch:

2. Bash

```
git clone https://gitlab.com/<username>/<repo>.git
```

```
cd <repo>
```

```
git checkout -b feature-update
```

3.

4.

5. Make changes, stage, and commit:

6. Bash

```
echo "Updated module documentation" >> README.md
```

```
git add README.md
```

```
git commit -m "Update documentation for merge request"
```

7.

8.

9. Push the feature branch to GitLab:

10. Bash

```
git push origin feature-update
```

11.

12.

13. Create Merge Request (MR) on GitLab:

- Open the repository page on GitLab.
- Click **Create Merge Request** (appears automatically after pushing).
- Select **feature-update** as the **Source Branch** and **main** as the **Target Branch**.
- Fill in the Title and Description, then click **Submit Merge Request**.

14. Review the Merge Request:

- Go to **Merge Requests** in the left sidebar on GitLab.
- Click on the submitted MR.
- Select the **Changes** tab to review diffs line-by-line.
- Leave inline comments or approve the MR.
- Click **Merge** to integrate changes into the target branch.

Question 2: Demonstration and execution of basic Docker commands.

1. Check Docker version and system information:

2. Bash

`docker --version`

`docker info`

3.

4.

5. Pull an official image from Docker Hub:

6. Bash

`docker pull nginx:alpine`

7.

8.

9. List local Docker images:

10. Bash

`docker images`

11.

12.

13. Run a container in detached mode with port mapping:

14. Bash

`docker run -d --name web_server -p 8080:80 nginx:alpine`

15.

16.

17. List active and all containers:

18. Bash

`docker ps`

`docker ps -a`

19.

20.

21. **View container logs:**

22. Bash

```
docker logs web_server
```

23.

24.

25. **Execute commands inside running container:**

26. Bash

```
docker exec -it web_server sh
```

27.

28. *(Type **exit** to exit the container shell.)*

29. **Stop and remove container:**

30. Bash

```
docker stop web_server
```

```
docker rm web_server
```

31.

32.

33. **Remove image:**

34. Bash

```
docker rmi nginx:alpine
```

35.

36.